

Astronautas

Atividade da Unidade 1 - C++ sem IA, depois com IA

Linguagem de Programação I

Sumário

Astronautas	1
1. A ideia	1
2. O contexto	2
3. Parte 1: o sistema, sem IA	2
4. Parte 2: mais funcionalidades, com IA	7
5. Entrega	10
6. Como será avaliado	11
7. Uso responsável	11
Instruções para a IA neste projeto	11
Diário da atividade	11
Ambiente	11
Parte 1: antes de programar	12
Parte 1: uso de IA para entender algo	12
Primeiro contato: revisão sem editar	12
Missão 1: LISTAR_ASTRONAUTAS e HISTORICO	12
Missão 2: SALVAR e CARREGAR	12
Missão 3: RELATORIO	12
Missão 4: livre	12
Fechamento	12

Astronautas

Atividade da Unidade 1 de Linguagem de Programação I.

1. A ideia

Você vai construir em C++ um sistema para uma agência espacial: cadastrar astronautas, cadastrar voos, lançar voos e registrar o que acontece com cada um. A atividade tem duas partes.

Na Parte 1 você programa sem IA. A ideia é praticar o que vimos nas aulas: classes, construtores, atributos privados, getters e métodos que mudam o estado do objeto. Também entra o vector, que a seção 3.3 explica. O sistema é pequeno e vem com testes prontos que dizem se está certo.

Na Parte 2 você usa uma IA de programação para acrescentar funcionalidades ao mesmo sistema. São três missões com objetivo definido e teste pronto, e uma missão livre. Aqui a ideia é você sentir o que a IA faz bem, onde ela erra, e aprender a pedir de um jeito que ela explique o que vai fazer antes de fazer.

Cada parte vale metade da nota.

O que você entrega é o link de um repositório público, no GitHub ou em outro serviço parecido. Crie o repositório com o conteúdo desta pasta e trabalhe nele desde o começo. O arquivo docs/diario.md é onde você registra o que pediu à IA e o que fez com cada resposta. Vou criar uma tarefa no SIGAA para você enviar o link. O prazo é o combinado com a turma e fica registrado nessa tarefa.

2. O contexto

A Agência Espacial Brasileira Digital cadastra astronautas e voos. Um voo começa planejado. Enquanto está planejado, astronautas podem ser adicionados ou removidos. Quando o voo é lançado, ele fica em curso e os astronautas a bordo ficam indisponíveis. Um voo em curso termina de duas formas: com sucesso, e os astronautas voltam a ficar disponíveis, ou com explosão, e os astronautas morrem.

Astronauta morto continua cadastrado, mas não embarca mais em voo nenhum. Se ele estava na lista de um voo ainda planejado, esse voo não pode ser lançado enquanto ele estiver lá.

O programa lê comandos, um por linha, até encontrar FIM. Assim dá para rodar o programa com um arquivo de comandos:

```
./agencia < testes/parte1/06_cenario_completo.in
```

Exemplo de arquivo de comandos:

```
CADASTRAR_ASTRONAUTA 111 30 Ana Maria
CADASTRAR_ASTRONAUTA 222 35 Bruno Costa
CADASTRAR_VOO 10
ADICIONAR_ASTRONAUTA 111 10
ADICIONAR_ASTRONAUTA 222 10
LANCAR_VOO 10
FINALIZAR_VOO 10
LISTAR_VOOS
LISTAR_MORTOS
FIM
```

3. Parte 1: o sistema, sem IA

3.1 O que já vem pronto

- `src/main.cpp`: o laço que lê os comandos já está escrito. Cada comando chega a um ponto marcado com `TODO`, onde você vai chamar o método certo.
- `testes/parte1/`: seis arquivos `.in` com comandos e seis `.out` com a saída exata esperada, do mais simples ao mais completo.
- `testes/testar.sh`: compila e roda todos os testes de uma pasta.

Antes de escrever qualquer linha, confira que tudo compila:

```
g++ -std=c++11 -Wall -Wno-sign-compare src/main.cpp -o agencia
./agencia < testes/parte1/01_cadastros.in
```

Nesse momento o programa imprime `TODO` para cada comando. É o esperado.

Sobre as opções: `-Wall` liga os avisos mais úteis do compilador. `-Wno-sign-compare` desliga um aviso que apareceria em todo laço escrito do jeito das aulas, com `int i` comparado com `size()`. Use esse comando na Parte 1. Se você separar o código em vários arquivos na Parte 2, o `README.md` mostra o comando novo.

Curiosidade: o `size()` devolve um número sem sinal, do tipo `size_t`, que nunca é negativo. O `int` tem sinal. O compilador avisa da mistura porque, na comparação, um `int` negativo viraria um número enorme. No laço da aula isso não acontece, porque `i` começa em 0 e só cresce. Se você quiser escrever sem o aviso, use o mesmo tipo dos dois lados:

```
for (size_t i = 0; i < cpfs.size(); i++) {    // size_t: sem sinal, como size()
    cout << cpfs[i] << endl;
}
```

Ou converta o tamanho para `int` na comparação: `i < (int) cpfs.size()`. Nesta atividade os dois jeitos estão bons.

3.2 As três classes

Você vai escrever três classes. Os atributos são `private`. `Astronauta` e `Voo` têm um construtor que recebe os dados do cadastro e inicializa os atributos. A `Agencia` começa vazia e não precisa de construtor. Todas têm métodos públicos para ler e para mudar o que precisa mudar.

Astronauta representa uma pessoa. Guarda:

- CPF (string);
- nome (string);
- idade (int);
- se está vivo (bool);
- se está disponível (bool).

Todo astronauta começa vivo e disponível. Quem muda esses dois valores é o próprio astronauta, por métodos como embarcar(), desembarcar() e morrer(). Ninguém de fora escreve nesses atributos.

Voo representa uma missão. Guarda:

- código (int);
- estado (string);
- CPFs dos astronautas a bordo (vector<string>).

Todo voo começa no estado planejado e sem astronautas. Os quatro estados possíveis, escritos exatamente assim, são:

```
planejado
em curso
finalizado com sucesso
finalizado com explosao
```

Agencia coordena tudo. Guarda um vector<Astronauta> e um vector<Voo>. Todas as regras da seção 3.4 ficam aqui: é a Agencia que confere se o astronauta existe, se o voo está planejado, se todos estão vivos, e só então manda o voo e os astronautas mudarem de estado.

Uma sugestão de esqueleto. Os nomes dos métodos são escolha sua. Os testes só olham a saída do programa. Aqui estão só as assinaturas. O corpo de cada método você escreve, do jeito que as aulas mostram.

```
class Astronauta {
private:
    string cpf;
    string nome;
    int idade;
    bool vivo;
    bool disponivel;
public:
    Astronauta(string cpf, string nome, int idade);
    string getCpf();
    string getNome();
    int getIdade();
    bool estaVivo();
    bool estaDisponivel();
    void embarcar();           // fica indisponivel
    void desembarcar();        // volta a ficar disponivel, se estiver vivo
    void morrer();             // fica morto e indisponivel
};

class Voo {
private:
    int codigo;
    string estado;
    vector<string> cpfs;
public:
    Voo(int codigo);
    int getCodigo();
    string getEstado();
    int getQuantidadeAstronautas();
    string getCpf(int posicao);
    bool temAstronauta(string cpf);
    void adicionarAstronauta(string cpf);
    bool removerAstronauta(string cpf);    // false se o CPF nao estava no voo
    void lancar();
    void explodir();
    void finalizar();
};
```

```
};

class Agencia {
private:
    vector<Astronauta> astronautas;
    vector<Voo> voos;
    int buscarAstronauta(string cpf);    // posicao no vector, ou -1
    int buscarVoo(int codigo);          // posicao no vector, ou -1
public:
    void cadastrarAstronauta(string cpf, string nome, int idade);
    void cadastrarVoo(int codigo);
    void adicionarAstronauta(string cpf, int codigo);
    void removerAstronauta(string cpf, int codigo);
    void lancarVoo(int codigo);
    void explodirVoo(int codigo);
    void finalizarVoo(int codigo);
    void listarVoos();
    void listarMortos();
};
```

Repare na divisão: o `lançar()` de `Voo` só troca o estado. Quem decide se o voo pode ser lançado é o `lançarVoo()` da `Agencia`. Se você colocar regra dentro de `Voo`, ele vai precisar conhecer os astronautas, e a classe cresce sem necessidade.

3.3 vector em meia página

Um vector funciona como um array que cresce durante a execução.

```
#include <vector>

vector<string> cpfs;           // vazio
vector<Astronauta> astronautas; // depois que a classe existir

cpfs.push_back("111");        // adiciona no final
int quantos = cpfs.size();    // quantidade atual
string primeiro = cpfs[0];    // acesso pelo indice, começa em 0

for (int i = 0; i < astronautas.size(); i++) {
    cout << astronautas[i].getNome() << endl;
}
```

Para remover, primeiro encontre a posição e depois chame `erase`. `cpfs.begin()` marca o começo do vector, e `+ i` anda `i` posições:

```
for (int i = 0; i < cpfs.size(); i++) {
    if (cpfs[i] == "111") {
        cpfs.erase(cpfs.begin() + i);    // remove a posicao i
        break;                          // ja removeu, sai do laço
    }
}
```

Uma dica sobre leitura: o nome do astronauta pode ter espaços. Por isso, no comando de cadastro, o nome vem por último e é lido com `getline(cin >> ws, nome)`. O `getline` lê até o fim da linha. O `cin >> ws` antes dele descarta os espaços e a quebra de linha que sobraram da leitura da idade. Isso já está feito no `main.cpp`.

3.4 Os comandos e as mensagens

Os testes comparam a saída do seu programa letra por letra. Por isso as mensagens estão fixadas aqui. Para cada comando, faça as verificações na ordem listada. A mensagem da primeira verificação que falhar é a que aparece, e nada mais acontece. Se nenhuma falhar, a operação acontece e a mensagem OK é impressa.

CADASTRAR_ASTRONAUTA cpf idade nome completo

- ERRO: astronauta com CPF 111 já cadastrado

- OK: astronauta 111 cadastrado

Repare que essa é a única mensagem com "com CPF". As outras dizem só "astronauta 111".

CADASTRAR_V00 codigo

- ERRO: voo 10 ja cadastrado
- OK: voo 10 cadastrado

ADICIONAR_ASTRONAUTA cpf codigo

- ERRO: astronauta 111 nao cadastrado
- ERRO: voo 10 nao cadastrado
- ERRO: voo 10 nao esta planejado
- ERRO: astronauta 111 esta morto
- ERRO: astronauta 111 ja esta no voo 10
- OK: astronauta 111 adicionado ao voo 10

Astronauta indisponível pode ser adicionado a um voo planejado. A disponibilidade só é conferida no lançamento.

REMOVER_ASTRONAUTA cpf codigo

- ERRO: astronauta 111 nao cadastrado
- ERRO: voo 10 nao cadastrado
- ERRO: voo 10 nao esta planejado
- ERRO: astronauta 111 nao esta no voo 10
- OK: astronauta 111 removido do voo 10

LANCAR_V00 codigo

- ERRO: voo 10 nao cadastrado
- ERRO: voo 10 nao esta planejado
- ERRO: voo 10 nao possui astronautas
- percorra os astronautas a bordo na ordem em que foram adicionados. No primeiro que estiver morto ou indisponível, imprima ERRO: astronauta 111 esta morto ou ERRO: astronauta 111 esta indisponivel e pare.
- OK: voo 10 lancado

Ao lançar, o voo passa para em curso e todos os astronautas a bordo ficam indisponíveis. Confira todos antes de mudar qualquer um: se o terceiro astronauta estiver morto, os dois primeiros não podem ter embarcado.

Confira morto antes de indisponível. Um morto também está indisponível, e a mensagem certa para ele é a de morto.

EXPLODIR_V00 codigo

- ERRO: voo 10 nao cadastrado
- ERRO: voo 10 nao esta em curso
- OK: voo 10 explodiu

Ao explodir, o voo passa para finalizado com explosao e todos os astronautas a bordo morrem.

FINALIZAR_V00 codigo

- ERRO: voo 10 nao cadastrado
- ERRO: voo 10 nao esta em curso
- OK: voo 10 finalizado com sucesso

Ao finalizar, o voo passa para finalizado com sucesso e os astronautas a bordo voltam a ficar disponíveis.

LISTAR_V00S

Mostra todos os voos agrupados por estado, na ordem dos quatro estados. Dentro de cada grupo, na ordem de cadastro. Cada voo mostra CPF e nome de cada astronauta a bordo, separados por vírgula, ou sem astronautas. Grupo vazio mostra (nenhum). O Voo só guarda CPFs. Quem acha o nome é a Agencia, procurando cada CPF no vector de astronautas.

Os dois exemplos abaixo são do mesmo cenário: os voos 10 e 40 terminaram com sucesso, o voo 20 explodiu com Carla e Diego a bordo, e o voo 30 ainda está planejado. Diego já tinha voado no 40 antes de morrer no 20.

```
LISTA DE V00S
== planejado ==
Voo 30: sem astronautas
== em curso ==
```

```
(nenhum)
== finalizado com sucesso ==
Voo 10: 111 Ana Maria, 222 Bruno Costa
Voo 40: 444 Diego Lima
== finalizado com explosao ==
Voo 20: 333 Carla Souza, 444 Diego Lima
```

LISTAR_MORTOS

Mostra os astronautas mortos, na ordem de cadastro, com os códigos dos voos dos quais participaram, na ordem de cadastro dos voos. Participou significa que o voo foi lançado com ele a bordo. Voo que ficou só planejado não conta.

```
ASTRONAUTAS MORTOS
333 Carla Souza - voos: 20
444 Diego Lima - voos: 20 40
```

Sem mortos: (nenhum). Morto que nunca voou: 333 Carla Souza - voos: nenhum. Esse último caso não aparece nos testes.

Comando que não existe: ERR0: comando desconhecido XYZ. Isso já está no main.cpp.

3.5 Como testar

Rode na raiz do repositório, a pasta onde está o README.md:

```
bash testes/testar.sh parte1
```

O script compila, roda cada .in e compara com o .out. Quando uma saída for diferente, ele mostra as linhas que divergem: as com - são o esperado, as com + são o que o seu programa imprimiu.

Os testes vão do mais simples ao mais completo. Faça um passar de cada vez. Se quiser testar uma situação que não está neles, crie o seu próprio arquivo de comandos e rode:

```
./agencia < meu_teste.txt
```

3.6 Antes de programar

Desenhe no papel as três classes como três caixas. Dentro de cada caixa, o que ela guarda. Entre as caixas, quem conhece quem: a Agencia conhece os dois vector; um Voo conhece os CPFs, e só os CPFs; um Astronauta não conhece voo nenhum.

Escreva em palavras o que acontece em LANCAR_V00 10: a Agencia procura o voo, confere o estado, confere se tem alguém a bordo, para cada CPF procura o astronauta e confere se está vivo e disponível, e só então manda cada um embarcar e manda o voo mudar de estado.

Preencha a seção "Parte 1: antes de programar" de docs/diario.md com o desenho que você fez, em palavras.

3.7 Ordem sugerida

1. Astronauta com construtor e getters. Agencia com o vector, buscarAstronauta e cadastrarAstronauta. Ligue o TODO de CADASTRAR_ASTRONAUTA. Compile.
2. Voo com construtor e getters. cadastrarVoo, listarVoos e listarMortos. Nesse ponto as listagens só vão mostrar voos sem astronautas e (nenhum), mas o teste 01 já precisa delas. O teste 01 deve passar.
3. adicionarAstronauta e removerAstronauta. O teste 02 deve passar.
4. lancarVoo e finalizarVoo. Teste 03.
5. explodirVoo. Testes 04, 05 e 06.

Compile depois de cada passo. Um erro por vez é mais fácil de achar.

3.8 Fechando a Parte 1

Quando os seis testes passarem, faça um commit com a mensagem Parte 1 pronta. A Parte 2 começa a partir desse commit, e na correção eu vou olhar o que existia nele.

Nesta parte a IA não escreve código. Você pode usar a IA para entender uma mensagem de erro do compilador ou um conceito da aula. Se fizer isso, anote no diário o que perguntou e o que aprendeu. Na conversa de avaliação eu vou pedir que você explique um trecho do seu código da Parte 1.

4. Parte 2: mais funcionalidades, com IA

4.1 A ferramenta: OpenCode

Três termos que aparecem daqui em diante:

- Modelo é o programa que gera texto e código a partir do seu pedido. Modelos diferentes respondem de forma diferente ao mesmo pedido. Alguns são gratuitos.
- Harness é o programa que fica entre você e o modelo: lê os seus arquivos, manda o pedido, recebe a resposta, aplica as mudanças e roda comandos. O OpenCode é um harness que funciona no terminal.
- Agente é o conjunto dos dois trabalhando em ciclo: o modelo decide o que fazer, o harness executa, o resultado volta para o modelo, até o pedido terminar. É assim que ele edita vários arquivos e compila sozinho.

Para instalar, siga as instruções de <https://opencode.ai/docs/>. Depois confira a versão instalada e abra o OpenCode dentro da pasta do seu repositório:

```
opencode --version
cd caminho/do/seu/repositorio
opencode
```

Dentro do OpenCode:

Comando	Para que serve
/connect	conecta um provedor; escolha OpenCode Zen
/models	lista e troca o modelo; escolha um marcado como gratuito
/new	começa uma conversa nova, sem o histórico anterior
/undo	desfaz a última resposta e as mudanças que ela fez nos arquivos
/exit	sai

O /connect pede uma chave. Para conseguir a chave, crie uma conta em <https://opencode.ai/auth>, copie a chave que aparece lá e cole no terminal. Os modelos marcados como gratuitos não cobram nada.

Para anexar um arquivo ao pedido, digite @ e o nome:

```
Leia @src/main.cpp e me diga o que cada classe guarda. Não altere arquivos.
```

Para rodar um comando e mostrar o resultado na conversa, comece a mensagem com !:

```
!bash testes/testar.sh parte1
```

Se a ferramenta pedir permissão para ler ou editar arquivos do repositório, aceite. Se pedir para instalar algo, acessar a rede ou mexer fora da pasta, recuse.

Não execute /init. Ele reescreve o AGENTS.md, que é o arquivo que a IA lê antes de cada pedido seu e que carrega as regras desta atividade.

Sobre o modelo: o Big Pickle normalmente está gratuito no OpenCode Zen e foi o que eu testei com esta atividade. Existem outros gratuitos na lista, e você pode experimentar. Anote no diário qual usou.

4.2 Primeiro contato: uma revisão sem editar

Antes da primeira missão, confira que a Parte 1 está commitada. Depois envie um pedido parecido com este:

Leia @AGENTS.md e @src/main.cpp. Não altere nenhum arquivo. Explique em poucas linhas o que o programa faz. Depois aponte três melhorias no código, explicando cada uma como se eu fosse iniciante e mostrando um exemplo curto.

Leia as três. É comum ela sugerir separar o código em vários arquivos, trocar o estado em string por outra coisa, ou marcar com const os métodos que só leem (uma promessa ao compilador de que aquele método não altera o objeto). Algumas sugestões vão contra o jeito que as aulas escrevem, como tirar o using namespace std. Você não é obrigado a aceitar.

Escolha uma sugestão que você entendeu e que faz sentido, e peça que ela aplique somente aquela. Depois rode `bash testes/testar.sh parte1`: os seis testes têm que continuar passando. Faça um commit e anote no diário o que aprendeu com essa primeira conversa.

Se você quiser separar o código em `include/` e `src/`, com um `.hpp` e um `.cpp` por classe, este é o momento. O script de teste aceita os dois formatos. Se fizer isso, atualize o comando de compilação no `README.md` e, nos pedidos das missões, troque `src/main.cpp` pelos nomes dos seus arquivos.

4.3 Como pedir: em dois tempos

Cada missão é um pedido em duas mensagens.

Na primeira você descreve o que quer e pede o plano, sem editar. Um pedido bom tem quatro partes:

1. O que existe: o que o programa faz hoje, em uma ou duas frases.
2. O que eu quero: o comando novo, a saída exata e as regras. Copie da missão.
3. O que não pode mudar: os comandos que já existem e a saída deles.
4. Como eu vou conferir: o teste da missão e o teste da Parte 1.

E termina com: "Antes de editar, me diga quais arquivos e quais métodos você vai criar ou alterar, e por quê."

Na segunda mensagem, se o plano fez sentido, você diz "pode fazer". Se não fez, você corrige o plano antes de qualquer linha ser escrita.

Exemplo para a Missão 1, primeira mensagem:

Este programa em C++11 controla astronautas e voos de uma agência espacial. Ele lê comandos da entrada padrão. As classes Astronauta, Voo e Agencia estão em `src/main.cpp`. Os testes em `testes/parte1` passam.

Quero dois comandos novos: `LISTAR_ASTRONAUTAS` e `HISTORICO cpf`. A saída exata está abaixo.

[cole aqui a seção da Missão 1]

Não mude nenhum comando que já existe nem a saída deles. Não use nada fora da biblioteca padrão.

Vou conferir com `bash testes/testar.sh missao1` e depois com `bash testes/testar.sh parte1`.

Antes de editar, me diga quais arquivos e quais métodos você vai criar ou alterar, e por quê.

Segunda mensagem, depois de ler o plano:

Pode fazer. Depois de editar, rode os dois testes e me mostre o resultado.

4.4 O ciclo de cada missão

pedir o plano -> ler o plano -> liberar -> ler o que mudou -> compilar
-> testar a missao -> testar a parte 1 -> anotar no diario -> commit

Para ver o que mudou, fora do OpenCode:

```
git status
git diff
```

Linhas com + entraram. Linhas com - saíram. Leia com atenção qualquer arquivo que você não esperava ver mudar.

Se o resultado não serve, desfaça com `/undo` dentro do OpenCode. Fora dele, `git checkout -- src` volta os arquivos de `src/` ao último commit (o mesmo vale para `include`, se existir). Arquivo novo que a IA criou aparece no `git status`, e você apaga à mão. Depois ajuste o pedido e tente de novo. Isso faz parte do trabalho.

Comece cada missão com `/new`, para a IA não misturar com a conversa anterior. Faça um commit ao terminar cada missão.

4.5 Missão 1: LISTAR_ASTRONAUTAS e HISTORICO

`LISTAR_ASTRONAUTAS` mostra todos os astronautas em três grupos, na ordem de cadastro dentro de cada grupo. No grupo `disponiveis` entram os vivos que não estão em nenhum voo em curso. No grupo `em voo` entram os vivos que estão em um voo em curso, com o código desse voo. No grupo `mortos` entram os mortos. Grupo vazio mostra (nenhum).

Os dois exemplos abaixo são do mesmo cenário: Ana voou no 10, que terminou com sucesso, e agora está no 20, em curso. Bruno morreu em outro voo.

```
LISTA DE ASTRONAUTAS
== disponiveis ==
333 Carla Souza (28 anos)
== em voo ==
111 Ana Maria (30 anos) - voo 20
== mortos ==
222 Bruno Costa (35 anos)
```

`HISTORICO cpf` mostra os voos dos quais o astronauta participou, com o estado atual de cada um, na ordem de cadastro dos voos. Participou significa que o voo foi lançado com ele a bordo. Voo só planejado não aparece. Sem voos: (nenhum voo). CPF desconhecido: `ERR0: astronauta 999 nao cadastrado`.

```
HISTORICO DE 111 Ana Maria
voo 10: finalizado com sucesso
voo 20: em curso
```

Teste: `bash testes/testar.sh missao1`. Depois: `bash testes/testar.sh parte1`.

4.6 Missão 2: SALVAR e CARREGAR

`SALVAR nome_do_arquivo` grava todos os dados em um arquivo de texto e imprime `OK: dados salvos em nome_do_arquivo`. Se não conseguir abrir o arquivo para escrita: `ERR0: nao foi possivel salvar em nome_do_arquivo`.

`CARREGAR nome_do_arquivo` substitui todos os dados atuais pelos do arquivo e imprime `OK: dados carregados de nome_do_arquivo`. Se o arquivo não existir: `ERR0: nao foi possivel carregar de nome_do_arquivo`, e os dados atuais continuam como estavam.

O formato do arquivo é escolha sua e da IA, desde que seja texto e que carregar depois de salvar reconstrua tudo: astronautas com vivo e disponível, voos com estado e lista de CPFs. Peça que a IA mostre o formato com um exemplo e explique como o programa reconstrói os objetos ao ler.

Teste: `bash testes/testar.sh missao2`. Ele roda três arquivos em sequência: o primeiro monta um cenário e salva em `dados_teste.txt`; o segundo, em outra execução do programa, carrega e continua operando; o terceiro tenta carregar um arquivo que não existe. Depois: `bash testes/testar.sh parte1`.

O script apaga `dados_teste.txt` no final. Para ver o arquivo, rode o primeiro teste à mão e abra o arquivo:

```
./agencia < testes/missao2/01_salvar.in
cat dados_teste.txt
```

É importante que você entenda o que está escrito lá. Na conversa de avaliação posso pedir para você explicar o formato.

4.7 Missão 3: RELATORIO

RELATORIO imprime a linha RELATORIO seguida de nove linhas sobre o estado atual:

```
RELATORIO
voos planejados: 0
voos em curso: 0
voos finalizados com sucesso: 1
voos finalizados com explosao: 2
astronautas cadastrados: 4
astronautas vivos: 2
astronautas mortos: 2
astronauta mais experiente: 111 Ana Maria (voos lancados: 1)
taxa de sucesso: 33%
```

Regras:

- A experiência de um astronauta é o número de voos já lançados em que ele estava a bordo. Voo ainda planejado não conta. Astronauta morto continua contando. Em caso de empate, vale o cadastrado primeiro. Se ninguém voou: astronauta mais experiente: (nenhum).
- Taxa de sucesso é a parte inteira de $\text{sucessos} * 100 / \text{finalizados}$, onde finalizados são os com sucesso mais os com explosão. Sem voos finalizados: taxa de sucesso: (nenhum voo finalizado).

Teste: `bash testes/testar.sh missao3`. Depois: `bash testes/testar.sh parte1`.

Dois testes desta missão usam a Missão 2, então faça a Missão 2 antes. O teste 04 monta um cenário, pede o RELATORIO e salva em `dados_teste.txt`. O teste 05, em outra execução, carrega esse arquivo e pede o RELATORIO de novo. A resposta tem que ser a mesma. Um erro comum aqui é guardar a experiência num contador dentro do astronauta e esquecer de gravar esse contador no arquivo.

4.8 Missão 4: livre

Escolha uma funcionalidade que você quer que o sistema tenha. Algumas ideias:

- limite de tripulantes por voo, informado no cadastro, e lista de espera quando o voo enche;
- comandante: um voo só pode ser lançado se tiver a bordo alguém com pelo menos dois voos concluídos com sucesso;
- destino e duração do voo, e um comando que mostra a agenda;
- tipos de voo com herança e método virtual, por exemplo voo de carga, que pode ser lançado sem astronautas, e voo tripulado, que não pode;
- comando DEMO que carrega um cenário de demonstração sem digitar nada;
- comando DESFAZER que volta o sistema ao estado anterior ao último comando;
- um menu interativo por cima dos comandos que já existem.

Não há teste pronto. Antes de pedir, escreva no diário o comando novo e a saída que você espera, e crie um arquivo de comandos seu para conferir. Só depois envie o pedido, em dois tempos como nas outras. No final, rode `bash testes/testar.sh parte1` de novo. A funcionalidade nova não pode quebrar o que já existia.

4.9 Quando a IA errar

Ela vai errar em algum momento. Código que não compila, teste que falha, mudança em arquivo que não devia mudar, recurso a mais que ninguém pediu, explicação bonita de algo que não funciona.

Quando acontecer, desfaça, anote no diário o que veio errado, melhore o pedido com o que faltou e tente de novo. A seção 6 explica como isso conta na nota.

5. Entrega

Envie na tarefa do SIGAA o link do seu repositório público. Ele precisa ter:

- o código-fonte compilando com o comando informado no README.md;
- o commit Parte 1 pronta no histórico;
- docs/diario.md preenchido com as suas palavras;
- os arquivos de comandos que você criou para a Missão 4;
- nenhum executável, arquivo de dados de teste ou arquivo temporário commitado.

6. Como será avaliado

Parte 1, metade da nota: os seis testes passam, as três classes têm as responsabilidades descritas, e você consegue explicar o seu código.

Parte 2, a outra metade: as missões 1 a 3 são conferidas pelos testes. A Missão 4 é conferida lendo o código e rodando o seu arquivo de comandos. O diário também conta: ele mostra o que você pediu, o que a IA planejou, o que você conferiu e o que decidiu. Uma missão com teste falhando, mas com o diário mostrando o que deu errado e o que você tentou, ainda recebe a parte do diário. Código que você não sabe explicar não conta.

Conversa de avaliação: cerca de cinco minutos comigo. Uma pergunta sobre o código manual, uma sobre uma mudança feita pela IA.

7. Uso responsável

- Use só dados fictícios. Não coloque nome ou CPF de pessoas reais.
- Não cole chaves, senhas ou tokens em prompt, código ou diário.
- Modelos gratuitos podem guardar as conversas. Não envie nada que não possa ser público.
- O código entregue é seu. Você responde por ele, inclusive pelo que a IA escreveu.
- Não esconda falha nem atribua à IA uma decisão que foi sua.

Instruções para a IA neste projeto

- Este é o projeto de um estudante iniciante em C++. Responda em português, de forma curta e didática. Quando explicar um conceito, use um exemplo pequeno.
- Use C++11 e somente a biblioteca padrão. Não instale nada, não acesse a rede, não use new, delete nem ponteiros. A única exceção é se o estudante pedir herança com método virtual na Missão 4; aí explique por que o vector precisa guardar ponteiros e quem libera a memória.
- Faça somente o que foi pedido nesta mensagem. Não adiante recursos que o estudante não pediu.
- Quando o estudante pedir um plano, não edite: diga quais arquivos e quais métodos você criaria ou alteraria, e por quê. Só edite quando ele liberar.
- Depois de editar, compile com o comando que está no README.md e rode `bash testes/testar.sh parte1` e o teste da missão pedida. Mostre o resultado real. Se um teste falhar, diga qual e por quê.
- Não altere os arquivos em testes/. Se um teste parecer errado, avise o estudante em vez de mudar o teste.
- Não mude a saída dos comandos que já existem.
- Não faça commit e não mexa no histórico do Git. Quem faz commit é o estudante.
- Não escreva o diário do estudante em docs/diario.md.

Diário da atividade

Escreva com as suas palavras. Frases curtas bastam. Não cole a conversa inteira com a IA. Cole só os pedidos que você enviou.

Ambiente

- Versão do OpenCode (`opencode --version`):
- Modelo usado:

Parte 1: antes de programar

- O que cada classe guarda:
- O que acontece em LANCAR_V00, em palavras:
- Uma dúvida que eu tinha antes de começar:

Parte 1: uso de IA para entender algo

- O que perguntei (ou “não usei”):
- O que aprendi:

Primeiro contato: revisão sem editar

- As três melhorias que a IA sugeriu, em uma linha cada:
- A que escolhi e por quê:
- O que mudou no código, e se os seis testes continuaram passando:
- O que entendi que não sabia antes:

Missão 1: LISTAR_ASTRONAUTAS e HISTORICO

- Primeira mensagem (o pedido do plano):
- O plano que a IA apresentou, resumido:
- Mudei algo no plano antes de liberar?
- Resultado de `testar.sh missao1` e de `testar.sh parte1`:
- Precisei refazer? O que mudou no pedido:

Missão 2: SALVAR e CARREGAR

- Primeira mensagem:
- O plano, resumido:
- O formato do arquivo (cole cinco linhas do `dados_teste.txt`):
- Resultado de `testar.sh missao2` e de `testar.sh parte1`:
- Precisei refazer? O que mudou no pedido:

Missão 3: RELATORIO

- Primeira mensagem:
- O plano, resumido:
- Resultado de `testar.sh missao3` e de `testar.sh parte1`:
- Precisei refazer? O que mudou no pedido:

Missão 4: livre

- O que escolhi e por quê:
- O comando novo, a saída que eu esperava e o nome do meu arquivo de comandos (escritos antes de pedir):
- Primeira mensagem:
- O que veio, comparado com o que eu esperava:
- `testar.sh parte1` continuou passando?
- Aceitei, ajustei ou descartei? Por quê:

Fechamento

- O que a IA fez que eu não conseguiria fazer sozinho nesse prazo:
- Onde ela errou ou fez algo que eu não pedi:
- O que eu faria diferente da próxima vez: